OASIS: Repairing Stale Optimizer Statistics in the Feedback Nullspace

Author Names Redacted for Review[a]

[a] *Affiliation Redacted for Review*

Abstract

Cost-based optimizers plan with column statistics that grow stale between refreshes, and query feedback exposes the resulting selectivity errors. Repairing a histogram from this feedback is an underdetermined problem. A handful of observed predicates constrain the marginal distribution only where they fall and leave the rest of it free, a region we call the feedback nullspace. Classical feedback-consistency methods such as STHoles, ISOMER, and QuickSel-H fill this nullspace with an uninformed default; ISOMER, for example, uses the maximum-entropy completion. The deployment question is then not how to match the feedback more exactly, but what to place in the nullspace.

We propose OASIS (Optimizer-Aware Statistics Imputation System), a statistics-layer middleware that imputes the nullspace with a completion learned against the optimizer's own error. OASIS is organized as three layers: a conversion layer that maps native engine statistics to a canonical distribution and back, a learned imputation layer that produces a marginal prior, and a calibration layer that projects the prior onto the feedback and routes among candidates using a deployment-visible signal. The prior is trained on a composite objective combining held-out future-predicate error, a join-regret term, feedback consistency, and a reconstruction regularizer, so that it reproduces the optimizer's behavior rather than the post-drift histogram. A rank analysis explains when the learned imputation helps: it improves accuracy in proportion to the nullspace the feedback leaves, and it reduces to the maximum-entropy default once dense feedback pins the marginal.

On held-out single-column drift, OASIS lowers future-predicate Q-error by 12.8% over ISOMER and also beats the self-tuning histograms STHoles and QuickSel-H (by 6.1% and 5.7%), with the gain reaching 21% on predicates farthest from feedback. A prior-source swap shows the calibration layer is estimator-agnostic. The corrected marginal stays safe downstream: it improves every member of a six-estimator composition family by 20–36%, repairs a FactorJoin join kernel on which an uncalibrated prior is harmful, and in a 12-configuration PostgreSQL planner study cuts row-estimation Q-error from 27.7 to 2.4 and new plan deviations from 2.3% to 1.1%, with the largest gap over ISOMER at sparse feedback (5.2× lower row Q-error at a two-predicate window). On a three-seed TPC-H SF10 study, the calibrated statistics match fresh-statistics accuracy and plans without a table scan, reproducing the fresh-statistics runtime to within 1.4× in the worst case; this is a deployment sanity check, not a claim of runtime improvement over leaving statistics stale.

Keywords: Statistics correction, Cardinality estimation, Histogram calibration, Cost-based optimizer, Query feedback, Underdetermined inverse problem

1. Introduction

Cost-based optimizers (CBOs) choose query plans from estimates of predicate selectivity, intermediate cardinality, and operator cost [1, 2]. In many analytical database systems these estimates rely on column-level statistics such as histograms, most-common-value lists, and density summaries [3, 4, 5, 6], refreshed periodically by `ANALYZE`-style procedures that read a full or sampled view of the table [7]. This maintenance strategy is simple and robust, but it leaves a persistent staleness gap: tables keep receiving inserts, deletes, and updates while the optimizer plans with the last statistics snapshot. As the gap widens, stale histograms induce systematic selectivity errors that propagate through the cost model into poor plans [8, 9, 10, 11].

A DBMS already obtains useful signal while executing ordinary queries. For a filter predicate, the optimizer's estimated selectivity and the executor's observed selectivity expose a local discrepancy between the stale histogram and the current data distribution. Feedback-driven self-tuning histograms turn such observations into statistics updates—STHoles, SASH, and ISOMER are the canonical examples [12, 13, 14, 15]—and ISOMER in particular makes the underlying principle explicit: among all marginals consistent with the observed feedback, choose the *maximum-entropy* one. This is a principled but uninformed choice: the least-committed completion of the part of the marginal that feedback does not determine.

Feedback repair is an underdetermined inverse problem.. The observation that organizes this paper is structural. An equi-depth histogram with B buckets has $B-1$ free interior quantile boundaries. A feedback window of K predicates constrains the cumulative distribution at the predicates' boundary values—at most one independent constraint per distinct boundary. When K is small relative to B, or when the observed predicates cluster, the feedback pins the marginal only along the span of those constraints and leaves the orthogonal complement—the *nullspace*—free. Any feedback-consistency method, ISOMER included, must *complete* the marginal in that nullspace, and the quality of an optimizer-facing statistic is governed by how well it does so on predicates that fall where feedback did not. Maximum entropy is one completion; it is not the completion that minimizes downstream optimizer error.

This reframing changes the deployment question from "how do we match the feedback better" to "what should we put in the nullspace." It also sets two boundaries that we make quantitative rather than rhetorical. First, a completion can only help where a nullspace exists: once feedback saturates the $B-1$ degrees of freedom, the marginal is pinned and *every* consistent method coincides. Second, a completion learned to reproduce the post-drift histogram—the obvious supervised target—does not help either, because reconstruction accuracy is not the same as optimizer relevance.

Contribution.. We propose OASIS (Optimizer-Aware Statistics Imputation System), which imputes the feedback nullspace with a learned, optimizer-relevant completion. OASIS is organized as three layers. A *conversion layer* maps native engine statistics to a canonical distribution view and back (Section 3.2). A *learned imputation layer* produces a marginal prior from a compact permutation-invariant model trained on a composite objective that scores the projected marginal by its held-out future-predicate error, a FactorJoin join-regret term, an in-window feedback-consistency residual, and a reconstruction regularizer (Section 3.3). A *calibration layer* reuses the classical feedback projection as a completion operator over any prior and adds a residual-gated router that selects between the learned completion and the maximum-entropy default from a deployment-visible signal (Section 3.4). Three findings support the design.

- The gain comes from the training objective rather than from calibration in the loop. An ablation that holds data, capacity, and projection fixed shows a reconstruction-only prior is worse than the classical priors at

every feedback budget, whereas the composite objective produces the gain. Applying the projection inside the training loop is not necessary, because post-hoc projection performs identically (Section 6.4).
- A rank analysis characterizes when learning helps. The improvement is governed by the nullspace the feedback leaves. It concentrates on predicates far from any observation (21% versus 9% near feedback), is already present at the sparsest feedback (4.3% over ISOMER at $K{=}2$), and reduces to a tie only where dense feedback saturates the constraints and pins the marginal, as in the planner regime (Section 4).
- The projection and router supply optimizer safety. A raw learned completion can over-extrapolate where the feedback does not pin it, which is harmful in a bilinear join kernel and on a real append trace. Projecting onto the feedback and routing on a deployment-visible residual make an imperfect prior safe to compose, join, and inject into a planner (Section 7).

Scope of the evaluation.. The evaluation is primarily *planner-facing*: we measure the corrected statistics object together with the row estimates and plan shapes a real PostgreSQL optimizer derives from it, plus one deliberately narrow TPC-H runtime check confirming that calibrated statistics reproduce fresh-statistics execution time. We do not claim broad query-runtime superiority over the stale baseline. We evaluate the mechanism where it acts—future selectivity, the feedback-locality structure the nullspace argument predicts, a six-estimator composition family and a FactorJoin join kernel that consume OASIS as a drop-in marginal source, and PostgreSQL planner estimates and plan shapes—and we probe external validity with out-of-distribution drift families, benchmark-inspired DML traces, and a public production HTTP telemetry trace used as an append-only event-table case study.

The contributions are:
- We formulate feedback-driven marginal repair as completion of an *underdetermined* system and give a rank characterization of the nullspace that predicts when a learned prior can help and when it must tie the maximum-entropy projection (Sections 2.4, 4).
- We train the nullspace completion on a composite optimizer-facing objective and show it beats both reconstruction-trained priors and classical self-tuning histograms wherever feedback under-determines the marginal; an ablation isolates the gain to the objective and shows in-loop projection is inessential (Sections 3, 6.4).
- We show the corrected marginal is safe to consume: it improves a six-

estimator composition family by 20%–36%, rescues a FactorJoin kernel where the raw prior regresses below stale, and reduces PostgreSQL new plan deviations to 1.1%, while a residual-gated router gives the same safety from a deployment-visible signal (Section 7).

- We give a primarily optimizer-facing evaluation spanning single-column drift, structural and out-of-distribution generalization, the composition family and FactorJoin kernel, a 12-configuration PostgreSQL planner-only study, a three-seed TPC-H runtime check, and sparse/noisy feedback diagnostics.

2. Background and Problem Formulation

2.1. A Deployment Scenario

Consider an analytical table such as TPC-H `lineitem` that accumulates inserts between scheduled `ANALYZE` passes. As rows shifted into a new date band arrive, the snapshot histogram of `l_shipdate` stops covering the current range, so the planner mis-estimates date-range predicates and can flip a join order or an index-versus-scan choice; queries that touch recent data then regress, sometimes sharply. Running `ANALYZE` would repair the statistics, but it reads a full or sampled view of the table and is therefore scheduled rather than continuous. In the meantime every executed query is a free measurement: for each filter conjunct the optimizer's estimated selectivity and the executor's observed selectivity reveal where the snapshot disagrees with the current data. The opportunity OASIS targets is to repair the optimizer-facing statistics from this feedback, between refreshes, without a scan.

2.2. Self-Tuning Histograms: the Framework We Build On

This goal places OASIS in the line of self-tuning histograms, whose design space is defined by three systems we also use as baselines. STHoles [13] builds and refines a histogram directly from query feedback, splitting buckets where the observed result contradicts the current estimate. ISOMER [15] adds a consistency principle: among all histograms consistent with the observed feedback it selects the maximum-entropy one, and it discards the oldest constraints when drift makes the active set inconsistent. QuickSel-H [16] fits a query-driven model to observed selectivities without touching the base data. All three share the interface OASIS adopts—feedback in, a corrected single-column statistic out, no rescan—and, as Section 2.4 formalizes, all three must *complete* the marginal in the region the feedback does not constrain.

OASIS keeps this interface and changes the completion rule, learning it against optimizer error instead of fixing it by maximum entropy or bucket splitting.

2.3. Column Statistics and the Staleness Gap

We model a column histogram as an equi-depth quantile summary with B buckets. The internal boundaries $v_1, \ldots, v_{B-1}$ correspond to fixed quantile levels $p_1, \ldots, p_{B-1}$, and a range predicate $x < c$ is estimated from the piecewise-linear CDF implied by those boundaries. Real DBMS statistics may combine histograms with most-common-value lists, density vectors, or null fractions; OASIS operates on a canonical one-dimensional CDF view and converts to and from native statistics through an engine-specific adapter (Section 3.2).

Let H_0 be the stale histogram at the last refresh and H^* the current post-drift distribution. For a predicate σ the optimizer derives an estimate $\hat{s}_{H_0}(\sigma)$ while the executor can later reveal an actual selectivity $s^*(\sigma)$. We measure error with Q-error,

$$\mathrm{QErr}(\hat{s}, s^*) = \max\left(\frac{\hat{s}}{s^*}, \frac{s^*}{\hat{s}}\right). \tag{1}$$

2.4. Statistics Correction as an Underdetermined Inverse Problem

Given a stale histogram H_0 and an observation window $\mathcal{O} = \{o_1, \ldots, o_K\}$, where each o_j records a predicate type, a boundary, an estimated selectivity, and an actual selectivity, the goal is corrected statistics $\hat{H}$ that lower expected future-predicate error,

$$\mathbb{E}_\sigma[\mathrm{QErr}(\mathrm{CDF}_{\hat{H}}(\sigma), s^*_\sigma)] < \mathbb{E}_\sigma[\mathrm{QErr}(\mathrm{CDF}_{H_0}(\sigma), s^*_\sigma)]\,,$$

the expectation taken over future predicates from the workload distribution. The solution must avoid table rescans (deriving $\hat{H}$ from H_0 and feedback only), run near planning time, and degrade gracefully when feedback is sparse.

The structure that drives this paper is a dimension count, which we state in the representation where the constraints are exactly linear. Fix the interval partition induced by the active feedback boundaries together with the B-bucket grid, and write the marginal as its vector of cell masses $m \in \Delta$, where Δ is the probability simplex over the cells (so $\dim \Delta = B - 1$ for the equi-depth grid). In this *bucket-mass representation* each feedback observation o_j constrains a *sum* of cell masses—the mass on its predicate interval equals the observed selectivity (two such sums for a `BETWEEN`)—so the window imposes a system of linear equality constraints $Am = y$. Let

$r = \text{rank}(A)$. The marginals consistent with the feedback form the affine slice $\{m \in \Delta : Am = y\}$, of free dimension $\dim \Delta - r$; we call this free part the *nullspace* the feedback leaves. The quantile boundaries a histogram exposes are recovered from m by inverting the implied CDF, a one-to-one map for a fixed grid.

Proposition 1 (Pinned regime)**.** *Fix the bucket-mass representation above and assume the feedback is consistent and noise-free, and that every method enforces it through the same linear constraints* $Am = y$*. If those constraints identify the marginal within this representation (full rank,* $r = \dim \Delta$*), then the consistent set* $\{m \in \Delta : Am = y\}$ *is a single point, and ISOMER, STHoles, QuickSel-H, and the projection of any prior all return the same marginal. A prior can differ from the maximum-entropy completion only on the nullspace of* A*, of dimension* $\dim \Delta - r$*.*

Proposition 1 is simple but central: the entire opportunity for a learned prior lives in the nullspace, and the opportunity vanishes continuously as feedback saturates the constraints. ISOMER fills the nullspace with maximum entropy [15]; STHoles and QuickSel-H fill it with their own structural assumptions. The question OASIS answers is what completion minimizes *downstream optimizer error*, and Section 4 measures the nullspace dimension as a function of K to predict exactly where any prior can act.

2.5. Scope

OASIS targets single-column histogram staleness—the foundational input that join estimators, composition models, and the planner all build on, as our FactorJoin and composition experiments make concrete. It does not repair multi-column correlation, join-key dependence, physical design, or cost-model calibration; these terms remain in the optimizer's error budget even when the single-column histogram is accurate. Improving the marginal that so many downstream estimators consume is a high-leverage target *when single-column staleness is a dominant error source*; where cross-column correlation or join-key skew dominates, an external dependence model is required and the gains here are correspondingly bounded. We treat this as a stated assumption, not a measured claim about any production system.

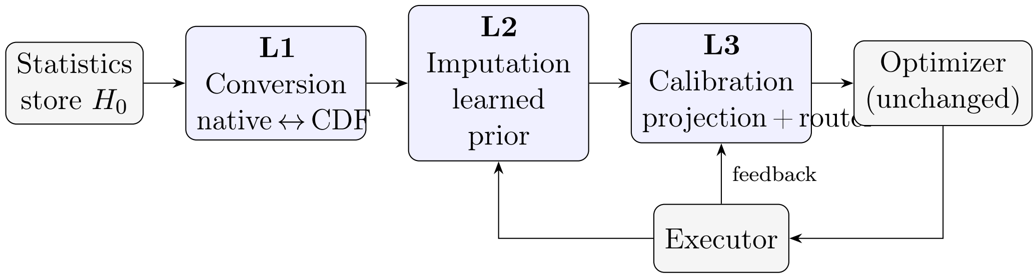

Figure 1: OASIS as a three-layer statistics middleware. A request for a histogram flows through the conversion layer (L1), the learned imputation layer (L2), and the calibration layer (L3), and the native object is returned to an unchanged optimizer. Predicate-level feedback from the executor conditions the imputation and drives the calibration. Only L2 is learned; L1 and L3 are deterministic.

3. OASIS Design

3.1. System Overview

OASIS is a middleware that sits at the statistics layer, between the statistics store and the optimizer, and is organized as three layers (Figure 1). The *conversion layer* (L1, Section 3.2) maps the engine's native statistics object to a canonical one-dimensional CDF view and back, so the rest of the system is engine-independent. The *imputation layer* (L2, Section 3.3) produces a marginal prior that completes the feedback nullspace; it is the learned component, and we refer to it interchangeably as Stage 1. The *calibration layer* (L3, Section 3.4) projects the prior onto the active feedback and routes among candidate marginals; we refer to it as Stage 2.

When the optimizer requests a histogram, the request passes down through the three layers and back: L1 reconstructs the canonical view, L2 supplies the prior from the stale view and a recent observation window, L3 calibrates it against the feedback, and L1 decomposes the result back into a native statistics object. Two integration points suffice: a query-completion listener that records predicate-level feedback from executor row counts, and a statistics-assembly hook that substitutes the corrected histogram before it reaches the optimizer. Because the correction is inserted at the statistics layer, the optimizer's search procedure and cost model are untouched, and the feedback collection adds only bounded per-conjunct bookkeeping.

Naming.. OASIS names the approach—the learned prior completed by feedback projection—and, as the "OASIS" column in result tables, its default deployed form: the learned prior under **hard projection**. **OASIS-noProj** is the raw prior with no projection. **Soft** is a consistency-gated soft projection and **Router** (also reported as the calibrated/aggressive Hybrid) is the residual-gated selector over {stale, ISOMER, OASIS-noProj, OASIS, Soft}. The external baselines are the **stale** histogram and **ISOMER** (the maximum-entropy projection initialized from stale), together with the self-tuning histograms **STHoles** and **QuickSel-H**.

3.2. Statistics-Format Conversion

Many engines do not expose a standalone editable equi-depth histogram. PostgreSQL, for example, couples histogram boundaries with MCV lists and other per-column summaries. We instantiate the interface for PostgreSQL-style statistics with a three-phase adapter: reconstruct a canonical distribution view from the native object, apply correction on the canonical view, and decompose the corrected view back into native fields. The round trip preserves total mass to machine precision (residual-quantile MAE mean 2.4×10^{-17}, max 9.2×10^{-17}), keeps selectivity estimates consistent (global MAE 1.1×10^{-4}, worst case below 0.39%), and completes in 0.066–0.073 ms.

Portability to other engines.. Only the conversion layer is engine-specific; the imputation and calibration layers operate on the canonical CDF and are unchanged across systems. Porting OASIS to a new engine therefore reduces to writing one adapter. The effort depends on how the engine exposes single-column statistics. MySQL and MariaDB keep equi-height/equi-width histograms in the data dictionary and expose them through `ANALYZE TABLE ... UPDATE HISTOGRAM` and `information_schema`, which maps to the canonical CDF almost directly. Microsoft SQL Server stores step-based histograms readable via `DBCC SHOW_STATISTICS`, again a close fit. Oracle couples height-balanced and hybrid histograms with column-level density and is closest to the PostgreSQL case we implement, where the adapter must also reconcile a most-common-value list. Engines that do not expose an editable histogram object would require correcting statistics through their maintenance API rather than in place. We validate the PostgreSQL adapter end-to-end; the others are design targets, not claims, but they share the same canonical interface.

3.3. Stage 1: A Prior Trained on a Composite Optimizer-Facing Objective

The Stage-1 component supplies the nullspace completion. The central design choice is the *training objective*, not the architecture: we train the prior so that, after feedback projection, the resulting marginal is a good optimizer input—not so that it reconstructs the post-drift histogram.

3.3.1. Representation and architecture

The prior generator consumes a fixed tensor with four blocks: normalized stale quantile boundaries, lightweight metadata, up to K recent predicate-feedback observations, and a validity mask. We use $B{=}10$ and $K{=}16$, small enough for planning-time use. The generator is a compact permutation-invariant set encoder over the stale-quantile and feedback tokens with a residual head that predicts corrections to interior quantile boundaries; the output is clamped and monotonized to a valid CDF. Permutation invariance over feedback tokens matches the unordered nature of an observation window and lets the same model consume any K.

3.3.2. Composite objective

Let p_θ be the prior marginal and Π the feedback projection. We train θ to minimize

$$\mathcal{L} = w_{\text{fut}}\,\text{QErr}_{\text{log}}\big(\Pi(p_\theta), \sigma_{\text{fut}}\big) + w_{\text{join}}\,\text{Regret}_{\text{FJ}}\big(\Pi(p_\theta)\big) + w_{\text{cons}}\,\rho_{\text{fb}}(p_\theta) + w_{\text{reg}}\,\|p_\theta - p_{\text{fresh}}\|_1, \quad (2)$$

where QErr_{log} is the held-out future-predicate log-Q-error, $\text{Regret}_{\text{FJ}}$ a differentiable FactorJoin bin-mass join regret, ρ_{fb} the in-window feedback residual, and the last term a light reconstruction regularizer toward the post-drift marginal. The first two terms make the objective *optimizer-facing*: the model is rewarded for the selectivity and join estimates the projected marginal induces, evaluated on predicates it has not seen, not for matching a histogram. Training uses a sparse-to-dense feedback curriculum and a domain-randomized mixture of drift families so the completion generalizes across feedback budgets and drift mechanisms; Section 6.4 shows that the reconstruction and consistency regularizers, not the optimizer terms alone, are what keep the dense-feedback regime from collapsing.

Two properties of this objective are worth noting. The projection Π appears inside the loss, which might suggest that "calibration in the loop" is the mechanism; the ablation in Section 6.4 shows otherwise, as post-hoc projection performs as well as in-loop projection. The reconstruction term is only a regularizer; a model trained on reconstruction *alone* is worse than the classical priors. The prior is trained offline and reused across columns without per-table retuning.

3.4. Stage 2: Projection as a Completion Operator and a Router

Stage 2 completes the prior in the feedback-constrained subspace and decides, per case, whether to trust the learned completion or fall back to the maximum-entropy one.

3.4.1. Hard projection (the completion operator).

The hard operator adapts the ISOMER/IPF feedback-consistency projection [15] to act on a supplied prior: it builds the interval partition induced by the active feedback predicates, initializes cell masses from the prior, and performs cyclic I-projections until every active predicate's interval mass matches its observed selectivity. When sequential drift makes the active set inconsistent, the oldest constraints are dropped first. The operator is not new; its role here is to realize the completion picture of Section 2.4—it pins the marginal exactly on the feedback span and leaves the learned prior to occupy the nullspace—and to provide the safe form for joins and the planner.

3.4.2. Soft projection and a banded negative control.

On future single-column predicates, matching a handful of recent intervals exactly can overfit; a soft operator that minimizes $\mathrm{KL}(p \,\|\, p_{\mathrm{prior}}) + \lambda \sum_j w_j (A_j p - y_j)^2$ over the same partition generalizes slightly better, with $w_j = \exp(-(\mathrm{conflict}_j/\tau)^2)$ down-weighting feedback that the most recent observations contradict. A banded relaxation of the equality constraints, by contrast, monotonically worsens accuracy as the band widens, so the soft operator's small edge comes from better cell-level conditioning, not from looser matching. We keep Soft as a router candidate but, given the ablation (Section 6.4), do not build the contribution on it.

3.4.3. The residual-gated Router.

No single operator dominates: hard projection is safest on joins and the planner, the learned completion wins on sparse single-column feedback. The Router selects, per case, the candidate with the lowest *in-window feedback residual*, the mean absolute discrepancy between a candidate's estimate and the observed selectivity computed over the *full* recent observation window.

This signal discriminates among candidates for two reasons, even though hard projection matches its active constraints exactly. First, the pool is not uniformly exact-matching: it includes the stale histogram and the raw prior (OASIS-noProj), which apply no projection and therefore leave a large residual, and the consistency-gated Soft candidate, which matches only approximately;

the residual separates these from the projected candidates cleanly. Second, the projection enforces only the *consistent active subset* of constraints—under sequential drift the stale-feedback filter drops the oldest constraints when the active set becomes inconsistent (Section 3.4)—so the full-window residual still measures how well each candidate fits the constraints just outside its own enforced set, where ISOMER (seeded from stale) and OASIS (seeded from the learned prior) differ. Among two candidates that enforce the same active set the residuals are nearly identical and ties are broken deterministically in favor of the safer projection. The gate never observes test predicates, true cardinalities, runtimes, or plan shapes, so it is non-oracle. Because it minimizes residual over a pool that contains the hard projection, it cannot increase the in-window residual relative to that pool; what this buys is a safe fallback wherever the learned completion over-extrapolates. Its per-budget choices follow the rank mechanism: it selects the learned completion where free degrees of freedom exist and reverts to ISOMER once the marginal is pinned.

4. When Does Learning Help? A Rank Characterization

Before the empirical ladder, we make Section 2.4's prediction quantitative, because it governs how every later result should be read. An equi-depth marginal with B=10 buckets has $B-1=9$ free interior boundaries. For each held-out case we count the independent feedback constraints (distinct interior boundary anchors, merged within tolerance) and the residual free degrees of freedom (DOF). Table 1 reports the mean rank and free DOF as a function of the feedback budget K.

The free DOF collapses from 6.68 at K=2 to ≈ 0 once the feedback constraints saturate the $B-1$ boundaries—here by $K \geq 12$ on this constraint-spanning predicate population, with 73% of marginals already pinned at K=8. The nullspace is therefore a property of *which regions of the domain the feedback constrains*, not merely of how many observations there are: a finite window pins the marginal near the observed predicates and leaves it free elsewhere. This predicts two signatures the rest of the paper confirms. (i) *Spatially*, the learned completion should win most on predicates that fall far from any feedback, directly in the nullspace, and least where the feedback already pins the marginal; the feedback-locality bins (Table 5) measure exactly this. (ii) *At saturation*, when a feedback window densely covers the queried region and drives the rank to $B-1$, the marginal is pinned and OASIS must

Table 1: Feedback-constraint rank and residual free degrees of freedom (DOF) of the marginal versus feedback budget K (marginal DOF = $B-1$ = 9). Free DOF is the nullspace dimension the projection leaves for the prior to complete; once feedback saturates the DOF the marginal is pinned and every consistent method coincides (Proposition 1).

K	mean rank	mean free DOF	marginals pinned
2	2.32	6.68	0%
4	4.65	4.35	0%
6	6.95	2.05	6%
8	8.71	0.29	73%
12	8.98	0.02	98%
16	8.98	0.02	98%

converge to ISOMER by Proposition 1; the dense PostgreSQL planner regime (Section 7.3) is where this tie appears. A prior given more feedback context fills the *residual* nullspace better, not less, so the completion's advantage persists—and even grows—across feedback budgets (Table 6) until the constraints actually saturate.

5. Experimental Methodology

The evaluation follows the deployment path of a marginal statistic. We first establish the corrected single-column object: the prior carries optimizer-relevant drift signal, its advantage is concentrated in the nullspace as Section 4 predicts, the gain comes from the composite objective, and the same calibration works for non-OASIS priors. We then widen from synthetic drift to out-of-distribution families, DML event streams, and a public telemetry trace. Finally we ask whether the corrected marginal stays safe once a composition estimator, a join kernel, or PostgreSQL's planner consumes it.

Unless stated otherwise, the prior is trained on a domain-randomized drift mixture and evaluated on held-out drift intensities $q \in \{1, 3, 5, 10, 15, 20, 25, 30\}$. All feedback-driven methods consume the same observation budget K=16 and emit corrected quantiles on the same B=10 grid. We compare against the stale prior, STHoles [13], ISOMER [15], and QuickSel-H [16], and against a recent learned query-driven baseline, LQM, which fits a monotone sigmoid-basis network to the same feedback window online, in the style of learned query-driven estimators [17, 18] but restricted to the single-column statistics-repair setting and the same no-rescan inputs. Every method

Table 2: Evaluation metrics. Direction marks whether lower (↓), higher (↑), or near-one ($\to 1$) is better. $\hat{s}, s^*$ are estimated and actual selectivity; p is the corrected marginal; $A_j p$ is its mass on feedback interval j with observed value y_j.

Metric	Definition	Dir.
Q-error	$\max(\hat{s}/s^*,\ s^*/\hat{s})$, Eq. (1)	↓
Selectivity MAE	mean $\lvert\hat{s} - s^*\rvert$ over predicates	↓
Quantile MAE	mean $\lvert v_i - v_i^{\text{fresh}}\rvert$ over boundaries	↓
Feedback residual	mean $\lvert A_j p - y_j\rvert$ on the observation window	↓
Row Q-error	Q-error of `EXPLAIN` estimated rows vs `COUNT(*)`	↓
Fresh-plan match	frac. of queries whose plan equals the fresh-stats plan	↑
Recovery	frac. of stale/fresh plan disagreements the method fixes	↑
New deviations	frac. of stale/fresh agreements the method breaks	↓
Join regret	selected-plan proxy cost / optimal proxy cost	↓
Time/Fresh	geomean warm-cache runtime ratio to fresh stats	$\to 1$

starts from the *same* stale statistics, sees the *same* feedback window, is projected onto the *same* constraints, and is evaluated on the *same* held-out split; no method receives oracle access, and "fresh" is computed from the post-drift data only as an accuracy upper bound. Table 2 defines the metrics used throughout. For sensitivity diagnostics we also use a transparent optimizer-decision proxy: future predicates are drawn from held-out distributions, each statistics state selects scan and join choices under a fixed cost proxy, and the choice is scored by its true proxy cost. The proxy analyzes feedback-budget and noise behavior; it is not a runtime benchmark.

Drift model and its justification.. We do not have access to a production stream of statistics-refresh histories, so we drive staleness with controlled drift, chosen to mimic how analytical tables actually evolve between `ANALYZE` passes. The drift intensity q scales how far the post-drift distribution moves from the snapshot H_0 (in units of inter-quantile shift), so larger q corresponds to a longer or more turbulent refresh interval; we sweep $q \in \{1, \ldots, 30\}$ to cover mild to severe staleness. The drift *families* (batch loads, range and date-band shifts, skew evolution, outlier bursts, multimodal and seasonal mixtures) are the update patterns that workload-evolution and learned-estimator drift studies report as the practical causes of estimator degradation [19, 20, 11, 21], and the DML traces of Section 6.7 reproduce several of them as explicit insert/update/delete streams. We treat the drift model as a stated,

Table 3: Projection-initialization ablation on held-out future predicates (in-distribution compound drift; geometric-mean selectivity Q-error). The projection operator and feedback constraints are identical across columns; only the marginal that initializes the projection changes. **ISOMER** initializes from the stale histogram and is exactly ISOMER; **OASIS** initializes from the learned stage and is the OASIS; **Fresh-init** initializes from the post-drift marginal as a reference upper bound. Win frac is the fraction of held-out predicates on which the OASIS beats ISOMER.

q	Stale	ISOMER	OASIS-noProj	OASIS	Fresh-init	Fresh	OASIS vs ISOMER	Win frac
1	1.209	1.142	1.174	1.148	1.104	1.000	-0.6%	50.7%
5	1.968	1.394	1.419	1.318	1.163	1.000	+5.4%	61.5%
10	2.990	1.617	1.464	1.372	1.176	1.000	+15.2%	70.1%
20	3.253	1.788	1.572	1.391	1.156	1.000	+22.2%	69.5%
30	3.193	1.779	1.624	1.429	1.159	1.000	+19.6%	68.9%
All	2.364	1.523	1.442	1.328	1.151	1.000	+12.8%	64.1%

Table 4: Projection-initialization with classical priors. The hard projection and feedback constraints are identical across columns; only the marginal that initializes the projection changes. **ISOMER** initializes from the stale histogram; **STHoles-init** and **QuickSel-init** initialize from those self-tuning-histogram priors; **OASIS** initializes from the learned MLP; **Fresh-init** is the post-drift upper bound. Last two columns are the OASIS-init improvement over the classical inits (positive favors OASIS).

q	Stale	ISOMER	STHoles-init	QuickSel-init	OASIS	Fresh-init	OASIS vs STHoles	OASIS vs QuickSel
1	1.209	1.142	1.147	1.165	1.148	1.104	-0.1%	+1.4%
5	1.968	1.394	1.344	1.358	1.318	1.163	+1.9%	+2.9%
10	2.990	1.617	1.554	1.528	1.372	1.176	+11.8%	+10.2%
20	3.253	1.788	1.565	1.484	1.391	1.156	+11.2%	+6.3%
30	3.193	1.779	1.510	1.540	1.429	1.159	+5.3%	+7.2%
All	2.364	1.523	1.415	1.407	1.328	1.151	+6.1%	+5.7%

transparent assumption rather than a claim about any single production system.

6. Single-Column Statistics Correction

6.1. The Learned Completion versus Classical Priors

We hold the projection operator and feedback constraints fixed and vary only the marginal that initializes the projection: stale (which yields ISOMER), the self-tuning histograms STHoles and QuickSel-H, the learned OASIS prior, and the post-drift fresh marginal as an upper bound. All variants are scored on held-out future predicates.

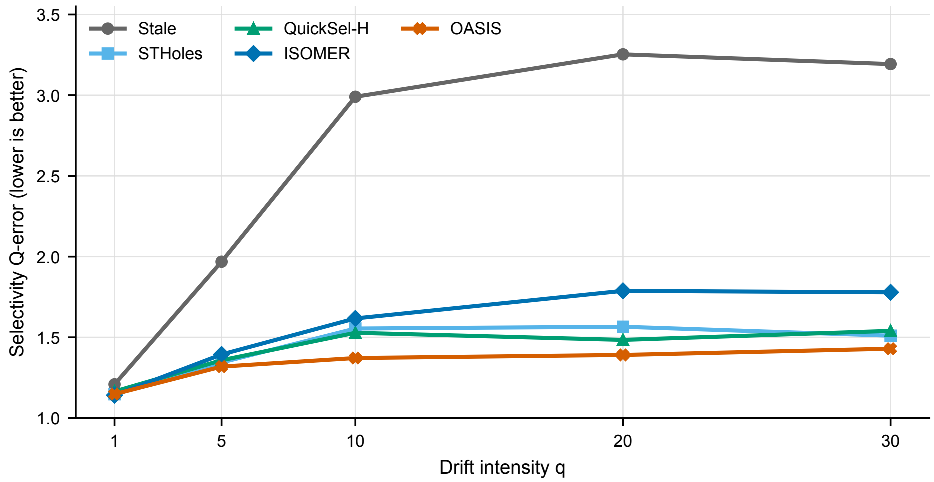

Figure 2: Single-column selectivity Q-error versus drift intensity for the deployable (feedback-projected) methods. Every prior is projected onto the same feedback constraints; OASIS is the learned completion. OASIS is the lowest non-fresh method at every drift level, and its margin over ISOMER and the self-tuning histograms widens as drift grows. Geometric means from Table 4.

Tables 3 and 4 and Figure 2 give the central single-column result. Initializing the same projection from the learned completion lowers aggregate future-predicate Q-error from 1.523 (ISOMER) to 1.328, a 12.8% reduction, and—unlike a reconstruction-trained prior—it also beats the classical self-tuning histograms, by 6.1% over STHoles (1.415) and 5.7% over QuickSel-H (1.407). The advantage grows with drift, reaching +15.2%, +22.2%, and +19.6% over ISOMER at q=10, 20, and 30, and the fresh-initialized lower bound (1.151) confirms headroom remains. That the learned completion beats two strong classical completions under the *same* projection shows that the gain is in what fills the nullspace, not in how the feedback is matched.

6.2. The Gain Lives in the Nullspace

Section 4 predicts that the learned completion should help most on predicates that fall where feedback did not. We test this directly by binning each held-out predicate by the directed Hausdorff distance from its endpoints to the nearest feedback endpoint in fresh-CDF space.

Table 5 and Figure 3 confirm the prediction. Near feedback (distance $\leq$ 0.05), where the projection already pins the marginal, OASIS beats ISOMER

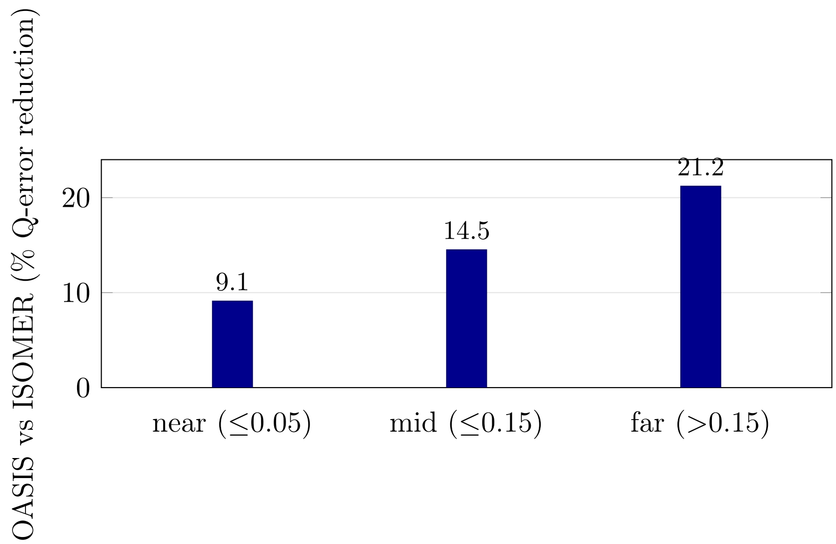

Figure 3: The learned completion's advantage over ISOMER grows monotonically with a held-out predicate's distance from the nearest feedback endpoint—the spatial signature of a nullspace completion. Near feedback, where the projection already pins the marginal, the gain is 9.1%; far from any feedback, squarely in the nullspace, it is 21.2%. Values from Table 5.

by 9.1%; in the mid band by 14.5%; and far from any feedback (> 0.15), inside the nullspace, by 21.2%. The improvement grows monotonically with distance from feedback, which is the spatial signature of a completion: the constraints determine the marginal near the observations, and the learned prior earns its advantage where they do not.

6.3. The Completion Persists Across the Feedback Budget

The complementary view varies the feedback budget K directly, in the optimizer-decision proxy, where future predicates are spread across the domain so that a finite window leaves residual nullspace at every budget.

Table 6 shows the learned completion beats ISOMER at *every* budget: +4.3% at K=2 (1.994 versus 2.083), +6.8% at K=4, +12.1% at K=8, and +15.7% at K=16 (1.402 versus 1.664). On this proxy surface ISOMER degrades gracefully as feedback thins, so the relative gap stays modest at every budget. On the real PostgreSQL planner the maximum-entropy completion instead fails sharply at sparse feedback, and there the gap is largest—a 5.2× row-Q-error difference at K=2 (Table 15). The two pictures agree on the

Table 5: Feedback-locality diagnostic on the mixed predicate population. Held-out future predicates are binned by the directed Hausdorff distance from their endpoints to the nearest feedback endpoint in fresh-CDF space. Values are geometric-mean selectivity Q-error; ISOMER is projection from a stale start and OASIS is the learned prior plus hard projection. OASIS win is the per-predicate fraction on which OASIS beats ISOMER.

Locality	Preds	Stale	ISOMER	OASIS-noProj	OASIS	Fresh	OASIS vs ISOMER	OASIS win
near$<=$ 0.05	10556	2.097	1.424	1.389	1.295	1.000	+9.1%	64.2%
mid$<=$ 0.15	6911	2.494	1.561	1.473	1.335	1.000	+14.5%	63.2%
far$>$ 0.15	3013	3.186	1.819	1.564	1.433	1.000	+21.2%	66.1%

Table 6: Feedback-budget sensitivity in the optimizer-decision proxy. All methods see only the K most recent feedback observations; OASIS uses the same fixed-width checkpoint with padding. Values are geometric means over all held-out predicates.

K	Stale QE	ISOMER QE	OASIS-noProj QE	OASIS QE	Hybrid QE	Aggressive QE	OASIS-noProj JoinOpt	OASIS JoinOpt	Hybrid JoinOpt	Hybrid choice
2	2.867	2.083	2.147	1.994	1.992	1.988	84.0%	85.4%	85.7%	ISOMER 31%, OASIS 66%, OASIS-noProj 2%, Stale 1%
4	2.867	1.797	1.924	1.675	1.665	1.658	86.7%	89.2%	89.3%	ISOMER 29%, OASIS 66%, OASIS-noProj 4%, Stale 1%
8	2.867	1.678	1.667	1.475	1.480	1.475	89.5%	91.8%	91.5%	ISOMER 31%, OASIS 59%, OASIS-noProj 8%, Stale 1%
16	2.867	1.664	1.543	1.402	1.425	1.428	91.0%	92.7%	92.2%	ISOMER 33%, OASIS 51%, OASIS-noProj 14%, Stale 1%

mechanism: the learned completion earns its advantage wherever feedback under-determines the marginal, and the methods converge only when dense feedback saturates the constraints and pins it. This is also why the gain concentrates spatially in the nullspace (Table 5) rather than at any particular budget.

6.4. The Gain Is the Objective, Not In-Loop Calibration

We now decompose the objective of Eq. (2), holding data, capacity, and projection fixed. Each variant's prior is calibrated by the *same* real projection on held-out future predicates and reported as percentage improvement over the strongest classical prior (STHoles) at three feedback budgets.

Three conclusions follow. First, reconstruction-only training fails at every budget: training a prior to look like the post-drift histogram makes it worse than the classical priors, so an optimizer-facing objective is necessary. Second, the composite objective creates the gain, and the reconstruction and consistency regularizers are essential at dense feedback: future-loss alone, and future-loss+join, collapse to a tie at K=16, and only the regularizers restore the dense-feedback margin. Third, *training the prior through the projection is not the source of the gain*: "raw-train + post-hoc projection," where the loss never sees the projection, matches "full in-loop" to within 0.5% at every budget, and the equivalence holds across two training seeds on the full drift range. We therefore do not claim in-loop calibration is necessary; it is a

Table 7: Stage-1 objective ablation (held-out future-predicate Q-error after the same real projection; % improvement over STHoles, higher is better). All variants share data, capacity, and projection. *Reconstruction-only* is the floor; the composite objective drives the gain; *raw-train + post-hoc projection* matches *full in-loop*, so differentiable projection *in the loop* is not the source of the gain. The dense-feedback (K=16) column shows the reconstruction/consistency regularizers, not in-loop projection, are what prevent collapse.

Stage-1 objective variant	K=2	K=6	K=16
Histogram reconstruction only	−1.2	−1.9	−7.8
Future-loss only (in-loop proj.)	+8.8	+7.0	+0.5
+ FactorJoin regret	+8.0	+5.9	−0.0
+ feedback residual	+9.8	+7.4	+3.4
Raw-train + *post-hoc* projection	+9.4	+9.0	+7.8
Full (in-loop projection)	+8.7	+7.9	+7.8

Table 8: Stage-1 prior-source swap under a fixed Stage-2 calibration protocol. Each row supplies a different single-column marginal prior to Stage 2; lower Q-error and feedback residual are better.

Stage-1 source	Raw QE	+Hard QE	+Router QE	Router gain	Raw resid.	Router resid.
Stale prior	2.404	1.539	1.437	40.2%	0.1206	0.0383
LinInterp	2.116	1.508	1.417	33.0%	0.1072	0.0379
FeedAvg	2.474	1.544	1.437	41.9%	0.1256	0.0383
STHoles	1.572	1.436	1.394	11.3%	0.0542	0.0365
QuickSel-H	1.614	1.423	1.392	13.7%	0.0573	0.0347
OASIS MLP	1.489	1.346	1.328	10.8%	0.0628	0.0354

convenient but inessential training device, and post-hoc projection—simpler to deploy—performs identically. The contribution is the composite objective.

6.5. The Calibration Is Estimator-Agnostic

The converse question is whether the completion-plus-projection picture is specific to the OASIS prior. We run a prior-source swap: the same projection and router are applied to stale statistics, two feedback heuristics (LinInterp, FeedAvg), STHoles, QuickSel-H, and the OASIS prior, on the same cached drift cases and feedback windows.

Table 8 shows the projection and router improve every prior, lowering future-predicate Q-error to a tight 1.39–1.44 band and the feedback residual to 0.035–0.038, and the Router is no worse than hard projection for *every*

Table 9: Out-of-distribution drift realism suite. OASIS is trained only on compound drift and evaluated on deployment-inspired drift families. Values are geometric mean selectivity Q-error over held-out cases; Hybrid uses only feedback-window residuals for selection.

Drift family	Stale	ISOMER	OASIS-noProj	OASIS	Soft	Hybrid	Aggressive	Fresh	Hybrid choice
Batch load	4.203	1.461	1.987	**1.251**	1.422	1.390	1.390	1.000	I 62%, P 38%, O 1%
Range shift	1.994	**1.210**	1.381	1.212	1.286	1.221	1.229	1.000	I 75%, P 22%, O 3%
Skew evolution	1.540	1.182	1.521	1.171	**1.163**	1.174	1.173	1.000	I 73%, P 27%, O 0%
Outlier burst	1.258	1.076	1.139	**1.064**	1.072	1.071	1.071	1.000	I 62%, P 37%, O 0%
Multimodal	1.238	1.066	1.136	**1.063**	1.065	1.065	1.069	1.000	I 70%, P 30%, O 1%
Seasonal/mixed	1.212	1.066	1.127	**1.058**	1.064	1.063	1.062	1.000	I 68%, P 32%, O 0%

source. The learned prior is now also the strongest *raw* input (1.489, below STHoles 1.572 and QuickSel-H 1.614), and it routes to the best final value (1.328). This restates the paper's central claim at the level of one experiment: the projection and router make any imperfect prior safe to use, while the trained completion is what wins the nullspace where the feedback leaves it open.

6.6. Out-of-Distribution Drift

We next leave the training generator. The prior, trained on the domain-randomized mixture, is evaluated unchanged on six deployment-inspired drift families: batch loads, range shifts, skew evolution, outlier bursts, multimodal drift, and seasonal/mixed drift.

Table 9 and Figure 4 show the expected graceful degradation. The deployed projected and routed forms remain the best non-fresh methods, edging ISOMER on batch-load (1.251 vs 1.461), skew (1.171 vs 1.182), and outlier drift; and where the *raw* prior over-extrapolates on an unseen family—batch-load (1.987) and skew (1.521)—the projection and residual-gated Hybrid pull it back to ISOMER level using feedback residuals alone. Under distribution shift the learned completion no longer dominates, but it never escapes the safety net.

6.7. Benchmark-Inspired DML Traces and a Public Telemetry Trace

Two further steps probe external validity. The first replaces drift operators with explicit DML event streams anchored to analytical maintenance patterns—append-heavy sales, promotion price revisions, inventory restocking, returns/cancellations, segment churn, seasonal maintenance—each emitting insert/update/delete counts before a feedback observation.

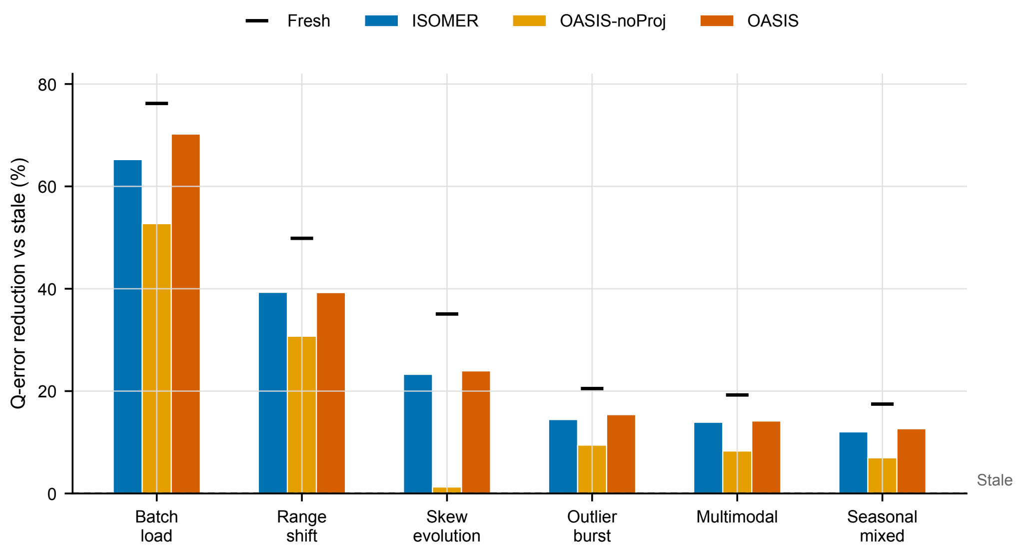

Figure 4: Out-of-distribution drift families: percentage reduction in geometric-mean selectivity Q-error relative to stale (higher is better; black ticks are the fresh reference). The projected OASIS matches or edges ISOMER on every unseen family, while the raw OASIS-noProj over-extrapolates—collapsing to near zero on skew evolution. The prior is trained only on the domain-randomized mixture. Values from Table 9.

The second uses a real public trace with a deliberately bounded claim: lacking access to private DBMS production logs, we use the NASA Kennedy Space Center HTTP request trace [22, 23], treating each request as an append to an event table and deriving three single-column signals (log reply bytes, time of day, log path length).

Tables 10 and 11, with Figure 5, reinforce the safety story. On the DML traces the Hybrid tracks ISOMER and stays the best non-fresh choice. On the real NASA trace the boundary is sharper: the raw learned completion regresses in aggregate (1.466 versus stale 1.241), and catastrophically on time-of-day predicates (2.66, worse than stale's 1.77), because a real append stream is not the training drift. The projection and residual-gated Router prevent the failure, recovering to ISOMER level (1.25 on time-of-day) and to the best deployable aggregate (1.115 for the Router). This is the deployment layer behaving as intended: trust the learned completion when the feedback window supports it, and fall back when the real trace exposes over-extrapolation.

Table 10: Benchmark-inspired DML trace sanity check. Each trace emits explicit insert/update/delete events anchored to an analytical benchmark table/column pattern. Values are geometric mean selectivity Q-error; no production trace or query runtime is used.

Trace	DML mix I/U/D	Growth	Stale	ISOMER	OASIS-noProj	OASIS	Soft	Hybrid	Aggressive	Fresh
Sales append	96/0/4	+79.9%	1.529	1.128	1.133	**1.114**	1.181	1.135	1.138	1.000
Promotion price	30/70/0	+33.9%	1.025	**1.018**	1.027	1.020	1.019	1.019	1.018	1.000
Inventory restock	19/75/6	+19.8%	1.222	**1.042**	1.127	1.046	1.054	1.047	1.050	1.000
Returns/cancel	38/17/45	-3.6%	2.047	1.163	1.161	**1.118**	1.187	1.170	1.172	1.000
Customer churn	36/62/2	+41.2%	1.172	**1.057**	1.159	1.079	1.084	1.061	1.063	1.000
Seasonal mixed	58/38/4	+50.8%	1.029	1.020	1.041	1.021	1.019	**1.019**	1.020	1.000

Figure 5: Benchmark-inspired DML traces: percentage reduction in geometric-mean selectivity Q-error relative to stale (cell value; positive is better, negative is a regression below stale). The projected OASIS and the Hybrid track ISOMER and the fresh reference, while the raw OASIS-noProj regresses on update-heavy traces (promotion price, seasonal mixed)—the failure the projection and router are there to absorb. Values from Table 10.

7. Optimizer-Facing Evaluation

We now ask whether the corrected marginal stays safe once a downstream estimator or a real planner consumes it. The boundary from Section 1 holds: the primary evidence is estimate quality and plan *shape*, with one narrow TPC-H runtime check.

7.1. Composition-Family Generalization

We embed OASIS as a drop-in marginal source in six independently designed composition estimators—the independence/maximum-entropy estimator, IPF/Sinkhorn on a 2D grid, and Gaussian, Clayton, Gumbel, and

Table 11: Public production-trace case study using the NASA Kennedy Space Center HTTP request trace. Each request appends one event to an analytics table; stale statistics are built from an early prefix, feedback is collected after later real requests, and future predicate probes include constants drawn from held-out later requests. Values are geometric mean selectivity Q-error. This is a public telemetry trace, not a private DBMS query log.

Trace column	Signal	Growth	Stale	ISOMER	OASIS-noProj	OASIS	Soft	Hybrid	Router	Fresh
Reply bytes	log reply size	800%	1.056	1.059	1.071	1.063	1.055	1.057	**1.053**	1.000
Time of day	request timestamp	800%	1.767	1.254	2.661	1.342	1.318	**1.251**	1.255	1.000
Path length	request path text	800%	1.025	1.057	1.104	1.082	1.065	**1.044**	1.050	1.000
Aggregate	all	–	1.241	1.120	1.466	1.156	1.140	**1.114**	1.115	1.000

Table 12: OASIS as a drop-in marginal upgrade across a family of composition estimators. Average joint Q-error (↓) for two-column range predicates; dependence structure is held fixed per estimator and only the marginal input varies. OASIS and Hybrid improve every estimator, whereas OASIS-noProj (the learned stage without the feedback-consistency projection) is weak and unstable, confirming the projection is required across composition methods.

Estimator	Stale	OASIS-noProj	ISOMER	OASIS	Soft	Hybrid	Aggr.	Fresh	OASIS-noProj +%	OASIS +%	Soft +%	Aggr. +%
Independence	1.924	1.913	**1.191**	1.231	1.278	1.203	1.202	1.138	+0.6%	+36.0%	+33.6%	+37.5%
IPF/Sinkhorn-2D	1.668	1.837	1.302	1.319	1.332	**1.292**	**1.291**	1.248	-10.1%	+20.9%	+20.2%	+22.6%
Gaussian copula	1.645	1.901	1.251	1.265	1.296	**1.238**	**1.237**	1.182	-15.6%	+23.1%	+21.2%	+24.8%
Clayton copula	1.836	1.949	1.310	1.300	1.333	**1.275**	**1.274**	1.222	-6.2%	+29.2%	+27.4%	+30.6%
Gumbel copula	1.639	1.833	1.254	1.271	1.297	**1.246**	**1.245**	1.192	-11.8%	+22.4%	+20.9%	+24.0%
Frank copula	1.621	1.899	1.291	1.302	1.329	**1.276**	**1.276**	1.221	-17.2%	+19.7%	+18.0%	+21.3%

Frank copulas—varying only the marginal input. The dependence structure is held fixed and identical for every marginal, so this is a clean marginal ablation, and the Archimedean copulas are intentionally mis-specified for the Gaussian-correlated data.

Table 12 shows the same pattern across the family. OASIS-noProj yields weak or negative gains, whereas OASIS improves every estimator by 19.7%–36.0% and the Hybrid by a comparable margin. The benefit does not depend on one dependence model and survives the mis-specified copulas: feedback consistency, not the raw prior, is the prerequisite for safe composition.

7.2. FactorJoin Integration

To test propagation into a learned *join* estimator, we embed OASIS in the bin-based join kernel of FactorJoin [24]. For an equi-join $A.k = B.k$ it estimates $\sum_{\text{bin}} \text{cnt}_A[\text{bin}]\,\text{cnt}_B[\text{bin}]/\text{dv}[\text{bin}]$, where each per-bin key frequency comes from a single-column key histogram, which we make the OASIS-correctable object. The keys are generated independently, so only the single-column key histogram varies across methods.

Table 13: OASIS embedded in the FactorJoin bin-based join kernel. Join-cardinality Q-error (↓) for a two-table equi-join whose join-key distributions drift; only the single-column key histogram consumed by FactorJoin varies across methods. OASIS and Hybrid recover most of the stale-to-fresh join error, whereas OASIS-noProj (the learned stage without projection) is actively harmful—it exceeds the stale baseline in this bilinear kernel.

q	Stale	OASIS-noProj	ISOMER	OASIS	Soft	Hybrid	Aggr.	Fresh	OASIS +%	Soft +%	Aggr. +%
5	1.323	1.484	**1.009**	1.015	1.032	**1.008**	**1.008**	1.005	+23.2%	+22.0%	+23.8%
10	1.179	1.699	**1.013**	1.028	1.059	1.015	1.015	1.005	+12.8%	+10.2%	+13.9%
20	1.179	1.673	**1.017**	1.030	1.053	1.019	1.019	1.005	+12.6%	+10.7%	+13.6%
30	1.178	1.638	**1.026**	1.037	1.040	1.027	1.027	1.005	+12.0%	+11.7%	+12.8%
All	1.213	1.621	**1.016**	1.027	1.046	1.017	1.017	1.005	+15.3%	+13.8%	+16.1%

Table 13 shows the strongest calibration failure mode. The raw completion is *actively harmful*, raising aggregate join Q-error to 1.621 versus stale 1.213, because the kernel is bilinear in per-bin mass and amplifies mass-concentration errors. Feedback projection fixes it: OASIS improves over stale at every drift level (+12.0%–+23.2%, +15.3% aggregate), the Hybrid by +16.1%, and both closely track the fresh marginal. The more nonlinear the consumer, the more essential the completion operator becomes.

7.3. PostgreSQL Planner-Only Statistics Injection

We validate that corrected statistics affect a real optimizer pipeline. The experiment builds alternative single-column statistics for `fact.x` and injects them into `pg_statistic`; true rows come from `COUNT(*)`, estimated rows and plan shapes from `EXPLAIN (FORMAT JSON)`. No query runtime is measured. The batch covers two drift directions, two table sizes, and three seeds (12 configurations).

Table 14 is the strongest DBMS-facing evidence, and its claim is deliberately one of *plan-shape safety*, not accuracy superiority over ISOMER. Pooling over all query instances (the aggregation quoted in this paragraph; Table 14 instead reports the per-configuration mean±sd, which differs slightly—e.g. stale row Q-error is 27.768 pooled versus 28.453 averaged over configurations), stale statistics give aggregate row Q-error 27.768 and match the fresh plan on 56% of queries. OASIS-noProj reduces row Q-error to 3.703 but introduces new plan deviations in 2.3% of cases. OASIS reduces row Q-error to 2.362, matches the fresh plan on 95.9% of queries, recovers 92.1% of stale/fresh disagreements, and cuts new deviations to 1.1%. Crucially, OASIS (2.362) and ISOMER (2.374) are near-identical here—and this is the rank mechanism, not a disappointment: these planner-facing windows are dense enough to pin

Table 14: Multi-configuration PostgreSQL planner-only statistics-injection evidence. Values are means over configurations; parentheses show one standard deviation across configurations. True rows use `COUNT(*)`; estimated rows and plan shapes use `EXPLAIN (FORMAT JSON)`. No query runtime is measured.

Method	Row QE	Fresh Plan	Recovery	New Deviations
Stale	28.453 (6.576)	56.1% (5.1)	0.0% (0.0)	0.0% (0.0)
ISOMER	2.390 (0.286)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
OASIS-noProj	3.894 (1.252)	90.9% (5.1)	80.7% (14.4)	2.4% (1.9)
OASIS	2.377 (0.269)	95.9% (1.9)	91.7% (3.8)	1.0% (1.0)
Soft	2.452 (0.312)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
Hybrid	2.391 (0.289)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
Aggressive	2.382 (0.283)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
Router	2.386 (0.288)	96.1% (2.0)	92.1% (4.1)	1.0% (1.0)
Fresh	1.415 (0.092)	100.0% (0.0)	100.0% (0.0)	0.0% (0.0)

the active constraints (Section 4), so hard projection converges to nearly the same marginal whichever prior seeds it, and the Router selects ISOMER on all twelve configurations. The planner is the regime where the *completion operator*, not the prior, does the work; the learned prior's advantage instead appears where the feedback leaves a nullspace, which on this same planner is the *sparse*-feedback regime we turn to next.

Sparse feedback on the real planner.. The 12-configuration batch above uses the standard dense window (K=16). To show the nullspace effect where it is sharpest, we re-run the same injection pipeline while varying the feedback budget K, over three drift configurations (Table 15). At K=2 the maximum-entropy projection collapses on the planner queries: ISOMER's aggregate row Q-error is 10.13 and it matches the fresh-statistics plan on only 63% of queries, because two constraints leave its completion far from the truth on the queried ranges. The learned completion holds at row Q-error 1.96 (Router 1.95) and a 94% fresh-plan match—a 5.2× accuracy gap on a real optimizer, not a proxy. As feedback densifies the constraints pin the marginal and the gap closes: by K=8 the two are within 3%, and at K=16 they tie (1.51 vs 1.53), recovering the dense-batch result above. This is the deployment knife-edge the paper's central claim predicts: on the planner, the learned imputation matters most exactly when feedback is too sparse to pin the statistics, and it costs nothing when feedback is dense.

Table 15: PostgreSQL planner-only injection under *sparse* feedback: aggregate estimated-row Q-error as the feedback budget K varies (means over three drift configurations; same `pg_statistic` path as Table 14). At K=2 the maximum-entropy projection (ISOMER) collapses while the learned completion (OASIS) and the Router hold; the methods converge as feedback densifies. Lower is better.

K	Stale	ISOMER	OASIS	Router	Fresh
2	27.9	10.13	**1.96**	1.95	1.17
4	28.1	1.72	**1.56**	1.56	1.16
6	28.0	1.82	**1.54**	1.52	1.16
8	27.7	1.57	1.52	1.52	1.16
16	28.0	**1.51**	1.53	1.52	1.17

7.4. TPC-H DML-Drift Runtime Sanity Check

Finally, a deliberately narrow runtime check, with its claim stated up front: calibrated statistics *reproduce fresh-statistics behavior*—accuracy, plan shape, and runtime—rather than beat the stale baseline on wall-clock. We load TPC-H SF10, drift it with an RF1-style insert stream that shifts `lineitem.l_shipdate` and `orders.o_orderdate` into a new date band, and inject single-column statistics for those columns exactly as above, reusing the *same* pretrained prior with no TPC-H retraining (also a cross-schema generalization test). For six date-sensitive queries we record scan row Q-error, plan shape, and warm-cache `EXPLAIN ANALYZE` time over three refresh-stream seeds. Because warm-cache runtime is determined by the induced plan, we time each distinct (query, plan) once at full repetition and assign it to every method sharing that plan.

Table 16 reports the result. Accuracy is unambiguous: stale scan Q-error 5.23 ± 0.24 is repaired to 2.02 for OASIS and 2.01 for the Router, matching fresh (2.01), on a standard schema with no retraining. Calibrated statistics reproduce the *fresh*-statistics runtime to within a few percent (time-to-fresh 0.96 for OASIS, 1.01 for the Router) because they induce the fresh plans. The *stale* wall-clock baseline is not a reliable target: here the stale mis-estimate happens to pick a cheaper-to-run plan, so calibrated and fresh statistics run $\approx 1.5\times$ slower than stale—the price of the correct plan, paid equally by fresh—whereas in a larger-drift configuration the same mis-estimate produced a plan $1.56\times$ slower than fresh. The deployment-relevant invariant is that calibrated statistics track fresh statistics in both accuracy and runtime, while stale statistics make the wall-clock outcome unpredictable in either direction. This

Table 16: TPC-H SF10 DML-drift study, aggregated over three independent refresh-stream seeds (mean $\pm$ std; six curated date-sensitive queries each; planner-only statistics injection, PostgreSQL 14). The pretrained OASIS prior is reused with no TPC-H retraining. *Scan Q-err* is the row-estimation error on the drifting date column. *Time/Fresh* and *Time/Stale* are geometric-mean ratios of warm-cache `EXPLAIN ANALYZE` execution time to the fresh- and stale-statistics runs. Calibrated statistics (OASIS, Router) match fresh statistics on both accuracy and runtime (Time/Fresh $\approx$ 1). The stale wall-clock baseline is not a reliable target: a stale mis-estimate can pick a coincidentally cheaper-to-run plan (Time/Stale $>$ 1 here), whereas in other configurations it picks a much slower one; we therefore make no runtime-superiority claim and report that OASIS reproduces fresh-statistics behavior.

Method	Scan Q-err	Time/Fresh	Time/Stale
Stale	5.23 ± 0.24	0.65 ± 0.07	1.00 ± 0.00
ISOMER	2.02 ± 0.08	0.98 ± 0.07	1.51 ± 0.07
OASIS-noProj	2.66 ± 0.07	1.02 ± 0.02	1.59 ± 0.18
OASIS	2.02 ± 0.10	0.96 ± 0.04	1.49 ± 0.11
Router	2.01 ± 0.10	1.01 ± 0.02	1.57 ± 0.15
Fresh	2.01 ± 0.07	1.00 ± 0.00	1.56 ± 0.17

is a single-schema check on six queries; we draw no broader runtime-superiority conclusion.

Tail safety.. Because a median or geomean can hide a bad tail, we report the runtime distribution against the deployment target, fresh statistics, over all 18 query instances (six queries, three seeds). Calibrated statistics induce the fresh plan on every instance—*zero* plan deviations from fresh for OASIS, the Router, and ISOMER—so there is no plan-shape tail. The runtime-to-fresh ratio stays tight: median 0.96 and worst-case 1.23 for OASIS, median 1.00 and worst-case 1.40 for the Router. The only large ratio appears against *stale*, on the single query (TPC-H Q14) whose plan changes stale$\rightarrow$fresh: under stale's coincidentally cheap plan it runs about 13$\times$ faster, but fresh statistics show the same $\approx$ 13$\times$ gap, so this is the cost of switching to the correct plan, not a regression introduced by calibration. No calibrated method is slower than fresh by more than 1.4$\times$ on any query or seed.

7.5. The Optimizer-Decision Proxy and Feedback Robustness

The proxy of Section 5 summarizes the regime structure on plan choices. Table 17 reports selectivity Q-error and join regret for each statistics state.

The ordering matches the rest of the paper: stale (2.867 Q-error, 1.187 regret) $\rightarrow$ ISOMER (1.664, 1.063) $\rightarrow$ the learned completion under projection

Table 17: Generator-driven optimizer-decision proxy. Q-Error and regret are geometric means; regret is true proxy cost of the plan selected from estimated statistics divided by the true optimal proxy cost (1.0 is optimal). No query runtime is measured.

Method	Sel. QE	Join Regret	Join Opt.	Fresh Match	Risk Resolved
Stale	2.867	1.187	79.0%	79.0%	0.0%
ISOMER	1.664	1.063	88.8%	88.8%	54.3%
OASIS-noProj	1.565	1.042	90.9%	90.9%	66.1%
OASIS	1.401	1.029	92.8%	92.8%	74.5%
Soft	1.362	1.025	93.2%	93.2%	75.3%
Hybrid	1.437	1.034	92.2%	92.2%	70.8%
Aggressive	1.441	1.035	92.1%	92.1%	70.1%
Router	1.375	1.028	92.9%	92.9%	74.0%
Fresh	1.000	1.000	100.0%	100.0%	100.0%

(1.401, 1.029) $\rightarrow$ the Router (1.375, 1.029, 92.9% join-optimal), approaching fresh (1.000). The Router improves on hard projection while never selecting a higher-residual candidate. Robustness along the remaining axes is consistent with the mechanism: the feedback-budget sweep (Table 6) is the rank curve, the per-intensity q columns probe drift severity, leave-one-family-out validation probes drift family (the prior trained on five families and tested on a held-out sixth beats the classical priors and ties ISOMER), and under 10% multiplicative feedback noise the Router degrades gracefully—selectivity Q-error rises only from 1.375 to 1.418 and join-optimal choices from 92.9% to 92.2%, still far below ISOMER (1.758) and stale (2.867). The failure modes are reported, not hidden: the raw prior is harmful in FactorJoin and on the NASA trace, which is why the projection and residual gating are part of the method.

8. Overhead and Deployment Considerations

Per-correction OASIS inference averages 1.06 ms (tensorization plus one forward pass); the PostgreSQL-style format conversion adds 0.066–0.073 ms, for a combined $\approx$ 1.13 ms per column. Feedback collection adds bounded per-conjunct bookkeeping and emits one observation tuple per conjunct at query completion. Maintaining these statistics therefore costs one forward pass plus the projection and avoids the full-table `ANALYZE` scan a refresh would incur—the intended use case is maintaining optimizer-facing statistics *between* scheduled refreshes, not replacing full refreshes after major schema changes or bulk shifts.

9. Related Work

Feedback-driven statistics and the completion question.. Cost-based optimization and selectivity estimation rely on compact statistics built from scans [1, 2, 3, 4], with streaming summaries such as KLL sketches providing approximate quantiles [25] and query feedback long used to adapt estimates [26]. Self-tuning histograms, STHoles, SASH, and ISOMER refine statistics from observed results [12, 13, 14, 15], and execution-feedback frameworks repair optimizer state over time [27]. OASIS shares this statistics-facing target but takes a different position: ISOMER's maximum-entropy projection is a *completion* of the feedback-underdetermined marginal, and we replace the uninformed completion with one trained against downstream optimizer error, bounded by a rank analysis of when any completion can help. This is, to our knowledge, the first account of feedback-driven repair as an underdetermined inverse problem with an explicit nullspace characterization.

Learned cardinality estimation.. Learned estimators such as NeuroCard, Naru, DeepDB, FACE, MSCN, FLAT, and Iris predict cardinalities from learned density or representation models [28, 29, 30, 31, 17, 32, 33], and benchmark and deployment studies document both their promise and the difficulty of robustness across workloads and drift [34, 35, 36, 10, 19, 20, 18, 37, 38, 21]. This line remains active in this journal: recent work learns dynamic sample selectors for cardinality estimation [39] and recursive-network models of complex predicates [40]. These systems output point estimates or estimator state; OASIS instead outputs corrected marginal *statistics* that an existing CBO consumes unchanged, and it trains the prior on the optimizer's error rather than on cardinality reconstruction, the same distinction the ablation draws internally.

Composition, joins, and plan-level learning.. Copula and factorized-join estimators such as FactorJoin [24] combine one-dimensional marginals with a dependence or join-key model; our composition-family and FactorJoin experiments embed OASIS as the marginal source in exactly these estimators and show feedback consistency is a prerequisite for safe downstream use. Plan-level and robust optimization systems—mid-query re-optimization, progressive optimization, LEO, Neo, Bao, and Flow-Loss [41, 42, 43, 44, 45, 46, 47, 11]—adjust plan choices or train plan-relevant estimates; OASIS operates below them by correcting the statistics object itself, and the two are complementary.

10. Limitations

The most important boundary is external validity. OASIS is evaluated on synthetic drift, benchmark-inspired DML traces, one public HTTP telemetry trace, and controlled PostgreSQL planner-only scenarios—not on a real DBMS query workload with its own refresh history. The NASA trace is real production telemetry but an append-only event-table proxy, not a database query log. Validating the same policy end-to-end on production statistics histories is the primary future direction. The mechanism is also intrinsically bounded: by Proposition 1, once feedback pins the marginal the learned completion provably cannot beat the projection and ties ISOMER, so the gains are concentrated where feedback leaves a nullspace—away from densely constrained regions—and we claim them only there. OASIS does not infer multi-column dependence from single-column feedback; where joint-cardinality error is dominated by cross-column correlation an external dependence model remains necessary. The TPC-H check shows only that calibrated statistics reproduce fresh-statistics execution time on one schema and six queries; runtime in general depends on executor behavior, caching, physical design, and cost-model calibration, and a stale mis-estimate can yield a coincidentally faster plan. Finally, the Router gates on feedback residual, a proxy for estimate quality rather than plan-shape risk: on the PostgreSQL subset it prefers ISOMER and inherits its slightly higher new-deviation rate, even though it never routes to Soft there. A plan-shape-aware gate is left to future work.

11. Conclusion

OASIS repairs stale single-column optimizer statistics from query feedback without rescanning tables or modifying the optimizer. Its organizing idea is that feedback repair is an underdetermined inverse problem: feedback pins the marginal only on the span of the observed predicates and leaves a nullspace that some completion must fill. The classical methods fill it with maximum entropy; OASIS fills it with a completion trained on a composite optimizer-facing objective (future-predicate error, join regret, feedback consistency, and a reconstruction regularizer), and a rank analysis describes where this can help.

On held-out single-column drift the learned completion cuts future-predicate Q-error by 12.8% over ISOMER and beats STHoles and QuickSel-H, with the gain rising to 21% on predicates farthest from feedback, present from the sparsest feedback (+4.3% at K=2) and persisting across budgets, while it provably ties ISOMER only where dense feedback saturates the constraints and pins the marginal. An ablation isolates the gain to the objective and shows in-loop projection is inessential. The same feedback projection and a residual-gated router make the imperfect completion safe to consume: OASIS improves a six-estimator composition family by 20%–36%, rescues a FactorJoin kernel where the raw prior is harmful, cuts PostgreSQL new plan deviations to 1.1%, and on TPC-H reproduces fresh-statistics accuracy and runtime without the table scan. OASIS is best understood as a reusable, feedback-calibrated statistics layer that learns the optimizer-relevant completion of the feedback nullspace and falls back to the maximum-entropy default where the nullspace closes.

References

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, T. G. Price, Access path selection in a relational database management system, in: Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, ACM, 1979, pp. 23–34. `doi:10.1145/582095.582099`.

[2] S. Chaudhuri, An overview of query optimization in relational systems, in: Proceedings of the 17th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, ACM Press, 1998, pp. 34–43. `doi:10.1145/275487.275492`.

[3] Y. E. Ioannidis, V. Poosala, Universality of serial histograms, in: Proceedings of the 19th VLDB Conference, 1993, pp. 256–267.

[4] V. Poosala, Y. E. Ioannidis, Selectivity estimation without the attribute value independence assumption, in: Proceedings of the 23rd VLDB Conference, 1997, pp. 486–495.

[5] PostgreSQL Global Development Group, PostgreSQL documentation: Planner statistics, `https://www.postgresql.org/docs/current/planner-stats.html`, accessed 2026-06-01 (2026).

[6] PostgreSQL Global Development Group, PostgreSQL documentation: Row estimation examples, `https://www.postgresql.org/docs/current/row-estimation-examples.html`, accessed 2026-06-01 (2026).

[7] PostgreSQL Global Development Group, PostgreSQL documentation: ANALYZE, `https://www.postgresql.org/docs/current/sql-analyze.html`, accessed 2026-06-01 (2026).

[8] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proceedings of the VLDB Endowment 9 (3) (2015) 204–215.

[9] G. Moerkotte, T. Neumann, G. Steidl, Preventing bad plans by bounding the impact of cardinality estimation errors, Proceedings of the VLDB Endowment 2 (1) (2009) 982–993.

[10] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cai, Y. Zhang, G. Li, Cardinality estimation in DBMS: A comprehensive benchmark evaluation, Proceedings of the VLDB Endowment 15 (4) (2021) 752–765.

[11] K. Lee, A. Dutt, V. R. Narasayya, S. Chaudhuri, Analyzing the impact of cardinality estimation on execution plans in microsoft SQL server, Proceedings of the VLDB Endowment 16 (11) (2023) 2871–2883. `doi:10.14778/3611479.3611494`.

[12] A. Aboulnaga, S. Chaudhuri, Self-tuning histograms: Building histograms without looking at data, in: Proceedings of SIGMOD, 1999, pp. 181–192.

[13] N. Bruno, S. Chaudhuri, L. Gravano, STHoles: A multidimensional workload-aware histogram, in: Proceedings of SIGMOD, 2001, pp. 211–222.

[14] L. Lim, M. Wang, J. S. Vitter, SASH: A self-adaptive histogram set for dynamically changing workloads, in: Proceedings of the 29th VLDB Conference, 2003, pp. 369–380. `doi:10.1016/B978-012722442-8/50040-9`.

[15] V. Markl, N. Megiddo, M. Kutsch, T. M. Tran, P. Lang, V. Raman, Consistent selectivity estimation via maximum entropy, The VLDB Journal 16 (2) (2007) 169–188.

[16] Y. Park, S. Zhong, B. Mozafari, QuickSel: Quick selectivity learning with mixture models, in: Proceedings of SIGMOD, 2020, pp. 1017–1033.

[17] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, A. Kemper, Learned cardinalities: Estimating correlated joins with deep learning, in: CIDR, 2019.

[18] P. Li, W. Wei, R. Zhu, B. Ding, J. Zhou, H. Lu, ALECE: An attention-based learned cardinality estimator for SPJ queries on dynamic workloads, Proceedings of the VLDB Endowment 17 (2) (2023) 197–210. doi:10.14778/3626292.3626302.

[19] B. Li, Y. Lu, S. Kandula, Warper: Efficiently adapting learned cardinality estimators to data and workload drifts, in: Proceedings of SIGMOD, ACM, 2022, pp. 2146–2160. doi:10.1145/3514221.3517914.

[20] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, M. Alizadeh, Robust query driven cardinality estimation under changing workloads, Proceedings of the VLDB Endowment 16 (6) (2023) 1520–1533. doi:10.14778/3583140.3583164.

[21] Y. Han, H. Wang, L. Chen, Y. Dong, X. Chen, B. Yu, C. Yang, W. Qian, ByteCard: Enhancing ByteDance's data warehouse with learned cardinality estimation, in: Companion of the 2024 International Conference on Management of Data, ACM, 2024, pp. 41–54. doi:10.1145/3626246.3653376.

[22] Internet Traffic Archive, NASA-HTTP: Two months of HTTP logs from the Kennedy Space Center WWW server, https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html, accessed 2026-06-01.

[23] M. F. Arlitt, C. L. Williamson, Web server workload characterization: The search for invariants, in: Proceedings of the 1996 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 1996, pp. 126–137. doi:10.1145/233013.233034.

[24] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, S. Madden, FactorJoin: A new cardinality estimation framework for join queries, Proceedings of the ACM on Management of Data (SIGMOD) 1 (1) (2023) 1–27.

[25] N. Ivkin, J. Nelson, H. Yu, V. Braverman, KLL: Approximately optimal streaming quantile estimation, Proceedings of the ACM on Management of Data 2 (1) (2019) 1–23.

[26] C. M. Chen, N. Roussopoulos, Adaptive selectivity estimation using query feedback, in: Proceedings of the 24th ACM SIGMOD International Conference on Management of Data, ACM Press, 1994, pp. 161–172. doi:10.1145/191839.191874.

[27] S. Chaudhuri, V. R. Narasayya, R. Ramamurthy, A pay-as-you-go framework for query execution feedback, Proceedings of the VLDB Endowment 1 (1) (2008) 1141–1152. doi:10.14778/1453856.1453977.

[28] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, I. Stoica, NeuroCard: One cardinality estimator for all tables, Proceedings of the VLDB Endowment 14 (1) (2020) 61–73.

[29] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, I. Stoica, Deep unsupervised cardinality estimation, Proceedings of the VLDB Endowment 13 (3) (2019) 279–292.

[30] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, C. Binnig, DeepDB: Learn from data, not from queries!, Proceedings of the VLDB Endowment 13 (7) (2020) 992–1005.

[31] J. Wang, C. Chai, J. Liu, G. Li, FACE: A normalizing flow based cardinality estimator, Proceedings of the VLDB Endowment 15 (1) (2021) 72–84. doi:10.14778/3485450.3485458.

[32] R. Zhu, Z. Wu, Y. Han, K. Zeng, A. Pfadler, Z. Qian, J. Zhou, B. Cui, FLAT: Fast, lightweight and accurate method for cardinality estimation, Proceedings of the VLDB Endowment 14 (9) (2021) 1489–1502. doi:10.14778/3461535.3461539.

[33] Y. Lu, S. Kandula, A. C. König, S. Chaudhuri, Pre-training summarization models of structured datasets for cardinality estimation, Proceedings of the VLDB Endowment 15 (3) (2022) 414–426. doi:10.14778/3494124.3494127.

[34] X. Wang, C. Qu, W. Wu, J. Wang, Q. Zhou, Are we ready for learned cardinality estimation?, Proceedings of the VLDB Endowment 14 (9) (2021) 1640–1654. doi:10.14778/3461535.3461552.

[35] J. Sun, J. Zhang, Z. Sun, G. Li, N. Tang, Learned cardinality estimation: A design space exploration and a comparative evaluation, Proceedings of the VLDB Endowment 15 (1) (2021) 85–97. doi:10.14778/3485450.3485459.

[36] K. Kim, J. Jung, I. Seo, W.-S. Han, K. Choi, J. Chong, Learned cardinality estimation: An in-depth study, in: Proceedings of SIGMOD, ACM, 2022, pp. 1214–1227. doi:10.1145/3514221.3526154.

[37] Z. Wang, Q. Zeng, N. Wang, H. Lu, Y. Zhang, CEDA: Learned cardinality estimation with domain adaptation, Proceedings of the VLDB Endowment 16 (12) (2023) 3934–3937. doi:10.14778/3611540.3611589.

[38] R.-A. Wang, Z. Zou, Z. Jing, Cardinality estimation via learned dynamic sample selection, Information Systems 117 (2023) 102252. doi:10.1016/j.is.2023.102252.

[39] R.-A. Wang, Z. Zou, Z. Jing, Cardinality estimation via learned dynamic sample selection, Information Systems 117 (2023) 102252. doi:10.1016/j.is.2023.102252.

[40] Z. Wang, H. Duan, Y. Cheng, G. Min, Learning complex predicates for cardinality estimation using recursive neural networks, Information Systems 124 (2024) 102402. doi:10.1016/j.is.2024.102402.

[41] N. Kabra, D. J. DeWitt, Efficient mid-query re-optimization of suboptimal query execution plans, in: Proceedings of SIGMOD, 1998, pp. 106–117.

[42] V. Markl, V. Raman, D. E. Simmen, G. M. Lohman, H. Pirahesh, Robust query processing through progressive optimization, in: Proceedings of SIGMOD, ACM, 2004, pp. 659–670. doi:10.1145/1007568.1007642.

[43] B. Babcock, S. Chaudhuri, Towards a robust query optimizer: A principled and practical approach, in: Proceedings of SIGMOD, ACM, 2005, pp. 119–130. doi:10.1145/1066157.1066172.

[44] M. Stillger, G. Lohman, V. Markl, M. Kandil, LEO – DB2's learning optimizer, in: Proceedings of the 27th VLDB Conference, 2001, pp. 19–28.

[45] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, N. Tatbul, Neo: A learned query optimizer, Proceedings of the VLDB Endowment 12 (11) (2019) 1705–1718.

[46] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, in: Proceedings of SIGMOD, 2022, pp. 1275–1288.

[47] P. Negi, R. Marcus, A. Kipf, H. Mao, N. Tatbul, T. Kraska, M. Alizadeh, Flow-loss: Learning cardinality estimates that matter, Proceedings of the VLDB Endowment 14 (11) (2021) 2019–2032. doi:10.14778/3476249.3476259.